

WASHERMANN

Financial & Pricing Module

Comprehensive Technical & Business Documentation

Version 1.0 · Confidential · May 2026

Audience: Engineering · Product · Finance · Investors · QA

Table of Contents

1. Executive Summary	3
2. Platform Overview & Architecture Philosophy	4
3. WashPoints — The Internal Currency	5
4. Conversion Rate System	7
5. Pricing Architecture	8
5.1 Platform Price List (Catalogue)	8
5.2 Pricing Engine	9
5.3 Fee Structure	11
5.4 All-In Pricing Model Endpoint	12
6. Special Pricing Packages	13
7. Vendor Price Intelligence Engine	15
8. Platform Configuration	17
9. Order Financial Flow (End-to-End)	19
10. Escrow System	21
11. Customer Wallet & Fiat Cost Basis Tracking	22
12. Vendor Earnings System	24
13. Rep Pseudo-Wallet & Bonus Tiers	25
14. Payout System	27
15. Gift Cards & Vaults	28
16. Audit Trail & Ledger Immutability	30
17. Profile Completion Gate	31
18. API Endpoint Reference	32
19. QA Test Scenarios	35
20. Glossary	38

1. Executive Summary

Washermann is a laundry-as-a-service marketplace connecting customers to vetted laundry vendors and delivery representatives ("Reps") in Nigeria. This document covers the complete financial and pricing architecture that powers every transaction on the platform.

The financial system is designed around four core principles:

- Predictability — customers always know what they will pay before confirming an order.
- Fairness — prices are informed by real vendor market data, not arbitrary markups.
- Flexibility — promotional pricing, package bundles, and audience-targeted offers can be deployed without engineering changes.
- Auditability — every penny movement leaves an immutable, timestamped record in a double-entry ledger system.

Key Financial Metrics (illustrative defaults):

Metric	Default Value	Configurable?
WashPoints per ₦1 (conversion rate)	Admin-set (e.g. 10 WP/₦1)	Yes — live
Service charge on order subtotal	5%	Yes — live
VAT	0% (can be enabled)	Yes — live
Rep share of order total	15%	Yes — live
Vendor payout rate	₦9 per WashPoint	Yes — live
Platform price offset over vendor median	25%	Yes — live
Price suggestion percentile (P70 default)	70th percentile	Yes — live
Order auto-complete (after delivery)	24 hours	Yes — live

i INVESTOR NOTE: All percentage values can be changed live by an admin without a deployment. Changes take effect on the next order placed — existing orders are never retroactively modified.

2. Platform Overview & Architecture Philosophy

2.1 Who the Participants Are

Participant	Role	Financial Relationship
Customer	Places laundry orders; pays in WashPoints	Buys WP with Naira via Paystack; WP debited on order placement
Vendor	Laundry business that processes garments	Receives WP earnings per completed order; cashes out to Naira bank account
Rep (Delivery)	Collects from customer, delivers to vendor and back	Earns WP per delivery logged in a pseudo-wallet; paid out as Naira salary each cycle
Admin / Finance	Manages platform config, approves payouts, monitors intelligence	Controls all fee structures, conversion rates, and price catalogue

2.2 Why a Point-Based Currency?

WashPoints (WP) are the internal unit of account for every transaction on Washermann. This design decision provides several advantages:

- Rate insulation

The platform can adjust the Naira↔WP conversion rate without touching price catalogue entries. A bag that costs 1,000 WP stays at 1,000 WP even if the Naira value changes.

- Loyalty potential

Points create natural stickiness. Customers who have accumulated WP are more likely to return.

- Multi-currency readiness

The internal WP layer can be priced against USD, GHS, or any other currency by simply adding a new ConversionRate row.

- Fraud surface reduction

WP cannot be withdrawn directly. They can only be spent on orders, gift cards, or package bundles, limiting exposure to financial fraud.

2.3 Architecture at a Glance

The system is built on NestJS 11 + TypeORM 0.3 + PostgreSQL. Every financial operation runs inside a database transaction to guarantee atomicity. No financial record is ever updated or deleted — only new records are appended.

Layer	Responsibility
PricingEngine (stateless)	Pure calculation — inputs in, PricingResult out. Zero database access.
PricingService	Loads config from DB, calls engine, orchestrates getClientConfig.

PricingIntelligenceService	Aggregates vendor prices, computes stats, suggests & applies platform prices.
PricingPackagesService	CRUD for promotional packages; audience-matching for customer-facing display.
PlatformConfigService	Single-row config store + immutable price list CRUD.
OrdersService	Authoritative order placement; wallet debit; escrow creation; earnings split.
Ledger entities	Append-only audit trail for customer wallets, vendor earnings, rep pseudo-wallets.

3. WashPoints — The Internal Currency

WashPoints (WP) are the exclusive unit of exchange inside Washermann. Every order, refund, gift card, and earnings payout is denominated in WP. Customers buy WP with Naira; vendors earn WP and cash out to Naira.

3.1 Acquiring WashPoints (Customer)

Customers top up their WashPoint balance by paying via Paystack:

1. Customer selects a WP amount to purchase on the app.
2. The system locks in the current conversion rate (e.g. 10 WP / ₦1).
3. A Paystack transaction record is created with the rate snapshot — this rate is frozen at initiation, not at payment completion, preventing rate-manipulation exploits.
4. Customer completes payment (card, bank transfer, USSD, etc.).
5. Paystack fires a webhook; the system verifies it independently.
6. WP are credited to the customer's Wallet and a LedgerEntry is written.

i IMPORTANT: The conversion rate is snapshotted at the moment the customer clicks "Buy WashPoints", not when the Paystack webhook arrives. This means a rate change mid-transaction does not give the customer more or fewer points than they bargained for.

3.2 Spending WashPoints

WP are debited from the customer's wallet at the moment an order is placed. The amount is determined by the authoritative server-side pricing calculation, never by a client-supplied figure.

3.3 Earning WashPoints (Vendors & Reps)

Vendors earn WP for each order they complete. These WP accumulate in a VendorEarningsWallet and can be requested as a Naira payout at any time.

Reps earn WP in a pseudo-wallet (not directly visible to them). Ops converts their accumulated cycle WP to a Naira salary each bonus cycle.

3.4 Fiat Cost-Basis Tracking (WACB Method)

The wallet entity stores both the WP balance and a parallel fiatBalanceKobo field. This tracks the real Naira value the customer has paid for their remaining WP, using Weighted Average Cost Basis (WACB) — the same method accountants use for stock portfolios.

How it works step by step:

7. Top-up: Customer pays ₦10,000 and receives 100,000 WP at 10 WP/₦1. fiatBalanceKobo increases by 1,000,000 kobo (= ₦10,000). WP balance = 100,000 WP.
8. Top-up again: Customer pays ₦5,000 and receives 45,000 WP at 9 WP/₦1 (rate changed). fiatBalanceKobo += 500,000 kobo. WP balance = 145,000 WP. Effective cost basis = ₦15,000 / 145,000 WP ≈ ₦0.103/WP.
9. Order placed for 50,000 WP. Proportion debited = 50,000 / 145,000 ≈ 34.5%. fiatBalanceKobo decreases by 34.5% × 1,500,000 = 517,241 kobo. Remaining balance = 95,000 WP, fiatBalance = 982,759 kobo (≈ ₦9,828).

This design enables:

- Tax reporting — the system can report how much Naira a customer actually spent
- Refund accuracy — refunds restore proportional Naira value, not an arbitrary WP amount
- Fraud detection — large discrepancies between WP balance and fiat balance flag anomalies

i TECHNICAL NOTE: WACB is simpler than FIFO lot tracking (which would require a separate lots table) and mathematically correct for tax purposes. A single column on Wallet is all that is needed.

4. Conversion Rate System

The ConversionRate table stores the official Naira-to-WashPoint exchange rate. Multiple rates can coexist; the "active" rate is the most recent one whose effectiveFrom is in the past.

4.1 How Rates Are Applied

Event	Rate Used	Behaviour
Customer buys WP via Paystack	Rate at initiation	Snapshot locked when transaction is created — not when webhook fires
Order pricing display (client)	Current active rate	Used to convert WP totals to Naira for display only
Order pricing (server, at placement)	Current active rate	Snapshot stored in pricingSnapshot on the Order record
Vendor earnings display	Rate at order time	nairaSnapshot stored in VendorLedgerEntry for historical reference
Vendor payout request	payoutRateNairaPerWP	Separate from the buy-rate; admin-configurable; snapshotted at request time

4.2 Buy Rate vs Payout Rate

There are two distinct rates in the system:

- Buy rate (pointsPerUnit on ConversionRate)

How many WashPoints the customer gets per ₦1 spent. Higher = more generous to customers = lower effective price.

- Payout rate (payoutRateNairaPerWP on PlatformConfig)

How many Naira a vendor or rep receives per WashPoint. This is the cost-of-goods-sold rate for the platform.

i INVESTOR NOTE: The spread between buy rate and payout rate is a key lever for platform profitability. Example: If customers buy at 10 WP/₦1 (₦0.10/WP) but vendors are paid at ₦0.09/WP, the platform retains ₦0.01 per WP on top of service charge. This spread, combined with the service charge, is the platform's primary revenue.

4.3 Multi-Currency Readiness

The ConversionRate entity has a currency field (default: "NGN"). Adding a second row with currency = "USD" immediately enables dollar-denominated WP purchases without any code change. The active rate is always the most recent approved entry per currency.

5. Pricing Architecture

Washermann pricing is built in three layers: a static price catalogue (what things cost in WP), a stateless calculation engine (what the customer actually pays), and an intelligent suggestion system (how prices stay competitive).

5.1 Platform Price List (Catalogue)

The PlatformPriceList table is an append-only catalogue of WashPoint prices. No entry is ever updated or deleted — price changes are implemented by adding a new row with a future effectiveFrom date. The system always uses the most recent approved row that has become effective.

Price entry types:

priceType	Key Fields	Example
bag	serviceType + bagSize	"wash_fold medium bag" = 1,200 WP
special_item	itemType	"suit" = 800 WP, "agbada" = 1,500 WP, "duvet" = 600 WP
ironing	(none — single global price)	"per garment ironing" = 80 WP

Bag size matrix (illustrative):

Bag Size	Wash & Fold (WP)	Wash & Iron (WP)	Approximate Naira (at 10 WP/₦1)
Small	800	1,100	₦80 – ₦110
Medium	1,200	1,600	₦120 – ₦160
Large	1,800	2,400	₦180 – ₦240
XL	2,500	3,200	₦250 – ₦320

All values above are illustrative. Actual prices are configured by admin.

Append-only rationale:

By never modifying existing price entries, the system preserves a complete price history. When auditing an old order, the exact prices in effect at that time can always be reconstructed. This also means pricing disputes can be resolved definitively.

5.2 Pricing Engine

The PricingEngine is a standalone, stateless TypeScript class. It has no database access and no NestJS dependencies. It accepts a PricingInputs object and a PricingConfig object, and returns a deterministic PricingResult.

Calculation Steps (in order):

Step	What Happens	Applied To
1. Bag price	Base price for service type + bag size	Every order
2. Special items	Per-item price × quantity for each requested item type	If customer adds special items
3. Ironing add-on	Per-garment iron price × ironing count	wash_iron orders only
4. Subtotal	Sum of steps 1–3	Derived
5. Service charge	$\text{subtotal} \times \text{serviceChargePercent} \div 100$	Every order
6. VAT	$(\text{subtotal} + \text{serviceCharge}) \times \text{vatPercent} \div 100$	When vatPercent > 0
7. Transport fee	Flat fee from area config (transportFeeWP)	Every order
8. Total	$\text{subtotal} + \text{serviceCharge} + \text{VAT} + \text{transport}$	Final charge to customer
9. Naira equiv.	$\text{totalWP} \div \text{pointsPerUnit}$ (display only, never stored)	For UI display

Full PricingResult object:

lineItems[]	– itemised breakdown (label, category, unitPriceWP, qty,
subtotalWP)	
subtotalWP	– sum before fees
serviceChargeWP	– calculated service charge
vatWP	– calculated VAT (0 when disabled)
transportWP	– area transport fee
totalWP	– final amount charged to customer
nairaEquivalent	– display-only Naira value
conversionRateId	– UUID of rate used
conversionRateSnapshot	– pointsPerUnit at calculation time
calculatedAt	– ISO 8601 timestamp

Why the engine is stateless:

The engine performs no I/O. This means it can be unit-tested exhaustively with zero mocking, runs identically on server and (if needed) on a client device, and has a provably deterministic output for any given inputs. The server runs it authoritatively at order placement; the client can run the same logic locally for real-time cart previews without a network call per update.

i SECURITY NOTE: The client is NEVER trusted to provide a final price. The server re-runs the full calculation at order placement. The client-side calculation is for UX speed only.

5.3 Fee Structure

Every order passes through several fee layers. Here is what each fee means and who it benefits:

Fee	Default	Applied On	Who Benefits	Configurable?
Service charge	5%	Order subtotal (items only)	Platform revenue	Yes
VAT	0%	Subtotal + service charge	Government / compliance	Yes
Transport fee	Varies by area	Flat per order	Covers Rep logistics cost	Per area
Rep share	15%	Total order value (informational)	Rep earnings pool	Yes
Platform spread	Implicit	Buy rate vs payout rate difference	Platform margin	Via rate config

Worked Example:

Customer orders a Medium Wash & Fold bag with one suit. Platform: 5% service charge, 0% VAT, ₦150 transport.

Line Item	WP	Naira (at 10 WP/₦1)
Medium Bag — Wash & Fold	1,200	₦120.00
Suit	800	₦80.00
Subtotal	2,000	₦200.00
Service charge (5%)	100	₦10.00
Transport	1,500	₦150.00
Total	3,600	₦360.00

5.4 All-In Pricing Model Endpoint (GET /pricing/model/:areald)

This endpoint returns the complete pricing model for a given delivery area. Its primary purpose is to give the mobile/web client everything it needs to show customers the final, all-inclusive price without a separate API call per cart change.

What "all-in" means:

Every item price is returned in two forms. The rawWP is the base platform price. The totalWP is the raw price after the service charge and VAT multiplier has been applied. Transport is excluded from item prices because it is a per-order flat fee, not a per-item charge.

```
// Example response structure (simplified)
{
  bagPrices: {
    wash_fold: {
```

```

        small: { rawWP: 800, totalWP: 840 }, // 5% SC applied
        medium: { rawWP: 1200, totalWP: 1260 },
        ...
    },
    specialItemPrices: {
        suit: { rawWP: 800, totalWP: 840 },
        agbada: { rawWP: 1500, totalWP: 1575 },
    },
    ironing: { rawWP: 80, totalWP: 84 },
    transportFeeWP: 1500,
    fees: {
        serviceChargePercent: 5,
        vatPercent: 0,
        repSharePercent: 15,
        feeMultiplier: 1.05,
        effectivePercentage: 5.0
    },
    conversion: { conversionRateId: "...", pointsPerUnit: 10, nairaPerWP: 0.1 },
    cachedAt: "2026-05-08T10:30:00.000Z"
}

```

The `feeMultiplier` field lets the client apply the same math to any custom item not in the list: $\text{customPrice} \times \text{feeMultiplier} + \text{transportFeeWP} = \text{total customer pays}$.

i IMPLEMENTATION NOTE: Clients should cache this response for approximately 5 minutes. The server will re-calculate authoritatively at order placement regardless of what the client cached.

6. Special Pricing Packages

Pricing Packages are promotional or niche bundles that exist outside the standard bag-size pricing system. They have a fixed WP price agreed at creation, a structured list of what is included, and powerful audience targeting rules.

6.1 Why Packages Exist

The standard bag-size model works well for general laundry but cannot express offers like "15 baby garments for ~~NY~~" or "5 corporate shirts pressed for ~~NY~~". Packages solve this by letting the admin define any arbitrary bundle with a fixed price, without needing to know how many bags it would equate to.

Example packages:

Package Name	What's Included	Fixed Price (WP)	Target Audience
Baby Bundle	15 baby garments, gentle wash & fold	8,000 WP	New parents; users with newborns
Corporate Pack	5 shirts + 3 trousers, wash & press	12,000 WP	Corporate company members
Duvet Season	2 duvets fully washed (limited time)	7,000 WP	All users (seasonal promo)
VIP Welcome	1 full-service order for new customers	5,000 WP	Users with 0 completed orders
Loyalty Reward	10% off any bag size (redeemed as package)	6,500 WP	Users with 10+ completed orders

6.2 Package Anatomy

Core fields:

- name — what the customer sees (e.g. "Baby Bundle")
- description — marketing copy describing the package
- imageUrl — artwork for the card in the app UI
- priceWP — the fixed WP price; not calculated from items
- criteria — structured list of what garments are included (for display; not enforced at the service layer in this version)
- audience — targeting rules (who can see this package)
- isActive — on/off toggle without deletion
- displayOrder — controls sort order in the app (lower = shown first)
- validFrom / validUntil — time-bounded promotional offers
- maxUsesPerUser — per-user usage cap (null = unlimited)

6.3 Audience Targeting

Each package carries an audience object that defines who can see and use it. All rules are AND-combined — a user must satisfy every condition that is set. Rules that are omitted or null are ignored (not restrictive).

Audience Rule	Data Source	Example Use Case
allUsers: true	—	Promo visible to everyone (overrides all other rules)
roles: ["corporate"]	user.roles	Packages only for corporate account holders
minOrderCount: 10	Count of completed orders	Loyalty rewards for frequent users
activeWithinDays: 30	Most recent completed order date	Re-engagement offers for lapsed users
arealds: ["uuid-lekki"]	Customer's order area history	Localised promotions for specific neighbourhoods
companyIds: ["uuid-X"]	CompanyEmployee membership	Corporate-exclusive packages for employees of a specific company
requirePhone: true	user.phone	Packages only available to verified phone users
requireAddress: true	Count of saved addresses	Packages that require a delivery address

How audience filtering works at runtime:

10. Customer hits GET /pricing/packages (authenticated).
11. All active, non-expired packages are loaded from the database.
12. For any package with allUsers: true, it is immediately included.
13. For others, the system builds an AudienceContext for the user: fetches user profile, address count, completed order count and dates, order area IDs, and company memberships — all in one parallel query batch.
14. Each package's audience rules are evaluated against the context.
15. Only packages where all conditions pass are returned to the customer.

i PRODUCT ROADMAP NOTE: The audience criteria system is designed to be extended. Future versions can add occupation-based targeting, loyalty tiers, referral status, or any other user attribute without changing the data model — just add new fields to the PackageAudience interface.

7. Vendor Price Intelligence Engine

The Intelligence Engine analyses what laundry vendors on the platform charge their customers for each type of garment, and uses that data to suggest optimal platform prices — ensuring the platform stays competitive while protecting its margin above the market median.

7.1 The Problem It Solves

Platform prices (bag prices, special item prices) are set by admin in the price catalogue. Without data, these are arbitrary guesses. The Intelligence Engine replaces guesswork with market evidence: if 50 vendors on the platform charge between ₦600 and ₦1,200 to wash a suit, the platform should price itself above the midpoint to protect margin — but not so far above that customers choose individual vendors over the platform.

7.2 How Data Is Collected

Each Vendor has a VendorPricing record containing their price list as an array of { garmentType, priceNaira } objects. Pricing updates are append-only; the most recent approved row per vendor is the "active" price. Vendors set prices in Naira; the system converts to WP for platform pricing.

7.3 Statistical Analysis

For each garment type that appears in at least one vendor's active price list, the engine computes:

Statistic	Meaning	Use Case
Vendor count	How many vendors price this garment type	Confidence indicator — ignore types with < 3 vendors
Min	Cheapest vendor on the platform	Floor — platform should never price below this
Mean	Average price across all vendors	Baseline reference
P25	25th percentile — 25% of vendors charge less	Budget end of the market
P50 (Median)	50th percentile — half above, half below	True market midpoint
P70	70th percentile — platform default suggestion	Above most vendors; protects margin
P75	75th percentile	More aggressive margin protection
P90	90th percentile — top of market	Premium positioning option
Max	Most expensive vendor on the platform	Ceiling reference

Why P70 as the default?

Setting the platform at the 70th percentile means the platform price is higher than 70% of individual vendors but lower than 30%. This is deliberately above the median because the platform provides

additional value: professional vetting of vendors, quality guarantees, contactless collection and delivery, dispute resolution, and loyalty rewards. The percentile is admin-configurable (priceSuggestionPercentile in PlatformConfig) so the positioning can be changed without code deployment.

7.4 From Naira to WashPoints

Vendor prices are stored in Naira. The suggestion is converted to WP using the current active conversion rate:

```
suggestedWP = round(suggestedNaira × pointsPerUnit)
```

For example: P70 = ₦900 Naira × 10 WP/₦1 = 9,000 WP

7.5 Applying Suggestions

The admin can review the full intelligence report at GET /pricing/intelligence and then apply suggestions via POST /pricing/intelligence/apply.

Apply options:

- garmentTypes — apply only for specific garment types (leave empty for all)
- toleranceWP — skip suggestions where the difference from the current price is $\leq$ this value (avoids noise updates for tiny changes)

Each applied suggestion creates a new entry in the platform_price_list table with priceType = "special_item" and effectiveFrom = now(). The existing entries are not deleted — they become historical records.

Intelligence Report Example:

Garment Type	Vendors	Min ₦	P50 ₦	P70 ₦	Suggested WP	Current WP	Diff WP
shirt	38	400	650	820	8,200	7,500	+700
suit	29	800	1,100	1,350	13,500	12,000	+1,500
agbada	21	1,200	1,800	2,200	22,000	22,000	0
duvet	17	500	800	950	9,500	10,000	-500
babygrow	8	200	320	390	3,900	0	+3,900

All values illustrative.

i INVESTOR NOTE: The intelligence engine is the platform's competitive moat. By continuously tracking vendor market prices, Washermann can ensure its pricing always reflects real market conditions — no manual research required.

8. Platform Configuration

All financial parameters live in a single, admin-editable configuration row. Changes take effect immediately on the next order — no deployment required.

Parameter	Type	Default	Effect When Changed
platformPriceOffsetPercent	Decimal %	25%	Baseline markup over vendor median — used as reference for Intelligence Engine
serviceChargePercent	Decimal %	5%	Immediately raises/lowers the fee charged on every new order subtotal
vatPercent	Decimal %	0%	Enables/disables VAT on all new orders (0 = disabled)
repSharePercent	Decimal %	15%	Controls how much of the order total is allocated to Rep earnings pool
payoutRateNairaPerWP	Decimal	₦9/WP	Changes the Naira value vendors/ reps receive per WP when cashing out
lowRatingThreshold	Decimal	3.5	Rating below which a Rep is auto-flagged for admin review
bonusCyclePeriod	Enum	monthly	Period over which Rep bonus earnings are accumulated before payout calculation
orderAutoCompleteHours	Integer	24 hours	Hours after delivery confirmation before order auto-completes if customer is silent
priceSuggestionPercentile	Integer	70	Which percentile of vendor prices the Intelligence Engine suggests as platform price

8.1 Guard Rails

Every parameter has validation constraints to prevent accidental misconfiguration:

Parameter	Min	Max
platformPriceOffsetPercent	0%	200%
serviceChargePercent	0%	30%
vatPercent	0%	30%
repSharePercent	0%	50%
lowRatingThreshold	1.0	5.0
orderAutoCompleteHours	1h	168h (1 week)
priceSuggestionPercentile	50	95

8.2 Who Can Change Config

Only users with the ADMIN role can update platform configuration. Finance role users can read it. All other roles have no access.

8.3 Audit Trail for Config Changes

The updatedBy field on PlatformConfig stores the UUID of the admin who last made a change. The updatedAt timestamp records when. The config table also has a createdAt. For full audit purposes, future versions can wrap config updates in a ConfigChangeLog table (the architecture supports this without model changes).

9. Order Financial Flow (End-to-End)

This section traces the complete money trail from the moment a customer places an order to the moment the vendor receives their earnings and can request a payout.

9.1 Pre-Order Checks

Before any financial operation begins, two gates are checked:

16. Profile Completion Gate — the customer must have a phone number and at least one saved delivery address. If either is missing, the order is rejected with a clear error listing the missing fields.
17. Balance Check — the customer's WashPoint balance must be $\geq$ the authoritative totalWP calculated by the server. If insufficient, the order is rejected.

9.2 Order Placement Transaction

All of the following happen inside a single atomic database transaction:

Step	What Happens	Database Records Created/Modified
1	Profile and balance pre-checks (outside transaction)	None yet
2	Authoritative pricing calculation via PricingEngine	None — pure computation
3	Customer wallet debited by totalWP	Wallet.balance decremented; Wallet.fiatBalanceKobo reduced proportionally (WACB)
4	Debit LedgerEntry written	LedgerEntry (type=debit, source=ORDER_DEBIT, amount, balanceBefore, balanceAfter, conversionRateId/Snapshot)
5	Order record created	Order (all fields; pricingSnapshot stores the full PricingResult JSON; status=PENDING_PAYMENT)
6	Escrow record created	OrderEscrow (holdWP = totalWP; locked until vendor completes; status=HELD)
7	Order status history entry	OrderStatusHistory (from=null, to=PENDING_PAYMENT)
8	Assignment broadcast created	AssignmentBroadcast (notifies available reps in the area)

9.3 Order Lifecycle & Earnings Release

Earnings are not released immediately. They follow the order through its lifecycle:

Status	Meaning	Financial Action
PENDING_PAYMENT	Order placed, awaiting Rep assignment	Funds in escrow

ACCEPTED	Rep accepted the broadcast	Funds in escrow
PICKED_UP	Rep has collected garments from customer	Earnings split calculated and locked into Order (vendorShareWP, repShareWP, platformFeeWP)
AT_VENDOR	Rep delivered to vendor; vendor processing	Funds in escrow
READY_FOR_PICKUP	Vendor finished; ready for rep to collect	Funds in escrow
DELIVERED	Rep returned clean laundry to customer	Funds in escrow
COMPLETED	Customer confirmed, or 24h auto-complete fired	Escrow released: vendor credited, rep pseudo-wallet credited, platform retains remainder
CANCELLED	Order cancelled (before PICKED_UP)	Full refund to customer wallet; fiatBalanceKobo restored proportionally

9.4 Earnings Split Formula

At PICKED_UP status, the order's earnings are locked:

```

vendorShareWP = round(totalWP × vendorSharePercent / 100)
repShareWP    = round(totalWP × repSharePercent / 100)
platformFeeWP = totalWP - vendorShareWP - repShareWP

// vendorSharePercent = 100 - repSharePercent - platformFeePercent
// These values are snapshotted at the time of PICKED_UP, not at order
completion.

```

i COMPLIANCE NOTE: Locking the split at PICKED_UP (rather than COMPLETED) prevents the platform from changing fee structures between a service being rendered and payment being released. This is important for regulatory compliance and vendor trust.

10. Escrow System

Every order's WashPoints are held in escrow from the moment of placement until the order is confirmed complete. This protects customers from paying for services not rendered, while assuring vendors they will be paid.

10.1 How Escrow Works

The OrderEscrow entity holds:

- holdWP — the exact WP amount held (= order totalWP)
- status — HELD, RELEASED, or REFUNDED
- releasedAt — timestamp when released to vendor
- refundedAt — timestamp if order was cancelled and funds returned

10.2 Escrow Release Conditions

Condition	Action	Who Triggers
Customer confirms order complete	Escrow RELEASED → vendor credited, rep credited	Customer (explicit confirmation)
24 hours pass after DELIVERED status	Auto-complete → escrow RELEASED → same as above	System (scheduled job)
Order cancelled before PICKED_UP	Escrow REFUNDED → full WP returned to customer wallet	Customer or Admin
Order cancelled after PICKED_UP (exceptional)	Partial or full refund depending on admin decision	Admin only

10.3 Why Escrow Matters (Investor Perspective)

Escrow is foundational to marketplace trust. Neither party can be defrauded: the customer cannot place an order they cannot pay for (balance checked first), and the vendor cannot be skipped on payment for work they have done. This trust mechanism is a key differentiator that enables Washermann to onboard professional vendors who would otherwise demand cash upfront.

11. Customer Wallet & Fiat Cost-Basis Tracking

11.1 Wallet Structure

Field	Type	Purpose
balance	bigint (WP)	Current spendable WashPoint balance
fiatBalanceKobo	bigint (kobo)	Running Naira cost basis of the WP balance (WACB method)
userId	uuid	One wallet per user

11.2 LedgerEntry — The Audit Trail

Every wallet change writes an immutable LedgerEntry:

Field	Purpose
type	"credit" or "debit"
amount	WP moved (always positive)
balanceBefore	WP balance before this event
balanceAfter	WP balance after this event
source	Why the movement happened (TOP_UP, ORDER_DEBIT, REFUND, GIFT_CARD_REDEMPTION, etc.)
conversionRateId	Rate active at this moment (for TOP_UP entries)
conversionRateSnapshot	pointsPerUnit frozen at the time
fiatAmountKobo	Naira paid (for TOP_UP entries only)
reference	Order reference or Paystack reference — traceable to the source event

11.3 LedgerSource Values

The source field uses a controlled enum. Current values:

Source	When Written
TOP_UP	Paystack payment completed and WP credited
ORDER_DEBIT	Customer places an order — WP debited
ORDER_REFUND	Order cancelled — WP returned
GIFT_CARD_REDEMPTION	Customer redeems a gift card code
VAULT_CREDIT	Admin or system credits WP directly from a vault
COMPANY_TRANSFER	Company wallet distributes WP to employee wallets

11.4 Why the Ledger Is Immutable

No ledger entry is ever modified or deleted in the service layer. This is a deliberate architectural choice that provides:

- Complete, tamper-proof financial history for every account
- Ability to replay all transactions and reconstruct any wallet balance at any point in time
- Regulatory compliance — financial records must be preserved
- Fraud detection — balance drift ($\text{wallet.balance} \neq \text{sum of ledger entries}$) immediately flags a discrepancy

12. Vendor Earnings System

Vendors earn WashPoints for every completed order. These accumulate in a dedicated VendorEarningsWallet separate from the main customer wallet. Vendors can request a Naira payout at any time.

12.1 VendorEarningsWallet Structure

Field	Purpose
balance	Current spendable WP balance — decremented when payout requested
totalEarned	All-time WP earned — never decremented (for lifetime stats)
status	"active" or "frozen" — frozen wallets cannot request payouts

12.2 VendorLedgerEntry

Like the customer ledger, all vendor wallet movements are append-only:

- VENDOR_EARNING — WP credited after order COMPLETED
- PAYOUT_DEBIT — WP debited when a payout request is processed

Each entry also stores nairaSnapshot — the Naira equivalent at the time, for the vendor's own records.

12.3 Vendor Pricing Data Feed

Vendors submit their own price list for each garment type (in Naira) via the VendorPricing system. This data feeds the Intelligence Engine (Section 7). Vendor pricing is append-only and requires admin approval before taking effect.

13. Rep Pseudo-Wallet & Bonus Tiers

Reps (delivery personnel) earn WashPoints for each delivery they complete. These are tracked in a "pseudo-wallet" — an internal ops tool, not visible to the Rep — and converted to a Naira salary + bonus each cycle.

13.1 Why a Pseudo-Wallet?

Reps are employees or contractors, not marketplace participants. They are paid a salary, not a withdrawal from a balance. The pseudo-wallet is an internal accounting tool that lets ops compute earnings precisely and audit them, while the Rep's external pay is a normal bank transfer.

13.2 RepPseudoWallet Fields

Field	Purpose
balance	Current cycle WP — reset to 0 at the start of each new bonus cycle
totalEarned	All-time WP earned — never reset
cycleStartedAt	When the current cycle began — used to calculate cycle duration

13.3 Bonus Tier System

At the end of each cycle, each Rep's average rating determines their bonus percentage. The bonus is applied to their cycle WP earnings before Naira conversion.

Tier	Rating Band	Bonus %	Flag for Review?
Elite	4.8 – 5.0	+15%	No
Gold	4.5 – 4.7	+10%	No
Silver	4.0 – 4.4	+5%	No
Bronze	3.5 – 3.9	0%	No
Review	0.0 – 3.4	0%	Yes — sent to admin review

Bonus tiers are fully admin-configurable via the `upsertBonusTier` endpoint. Multiple tiers can be active simultaneously; the system picks the one matching the rep's rating at cycle end.

13.4 Bonus Calculation Formula

```
cyclePayNaira = (cycleWP × payoutRateNairaPerWP) × (1 + bonusPercent / 100)
```

```
// Example: Rep earned 5,000 WP in the "Gold" tier (10% bonus)
```

```
// payoutRate = ₦9/WP
```

```
// cyclePayNaira = (5,000 × 9) × 1.10 = ₦45,000 × 1.10 = ₦49,500
```


14. Payout System

Vendors can request a Naira payout of their accumulated WashPoint earnings at any time. Payouts are processed via Paystack transfers to the vendor's registered bank account.

14.1 Payout Request Flow

18. Vendor submits a payout request (amountWP, bankCode, accountNumber, accountName).
19. System calculates $\text{nairaAmount} = \text{amountWP} \times \text{payoutRateNairaPerWP}$. The current payout rate is snapshotted into the request — rate changes after submission do not affect pending requests.
20. VendorEarningsWallet balance is debited immediately.
21. Request enters PENDING status.
22. Admin reviews and approves.
23. System initiates Paystack transfer.
24. Paystack webhook updates request to COMPLETED or FAILED.
25. On FAILED, the WP are returned to the vendor's earnings wallet.

14.2 Payout Request Fields

Field	Purpose
amountWP	WashPoints being cashed out
nairaAmount	Naira to be transferred (locked at request time)
payoutRateSnapshot	Rate used for this request (never changes after creation)
bankCode / accountNumber / accountName	Destination bank details
status	PENDING → PROCESSING → COMPLETED or FAILED
paystackReference	Paystack transfer reference for reconciliation
batchId	Optional — links this request to a bulk payout run
approvedBy / approvedAt	Admin who approved and when

i INVESTOR NOTE: The payout rate (payoutRateNairaPerWP) is intentionally separate from the customer buy rate (pointsPerUnit on ConversionRate). The buy rate determines how many WP customers get per Naira paid in. The payout rate determines how many Naira vendors receive per WP cashed out. The spread is platform margin.

15. Gift Cards & Vaults

Gift Cards allow WashPoints to be distributed to customers through promotional codes. Vaults are the pools of WashPoints from which gift cards are funded.

15.1 Vaults

A Vault is a pre-funded pool of WashPoints created by admin. Think of it as a budget allocation for a specific promotional campaign or purpose.

Vault Field	Purpose
purpose	GENERAL, GIFT_CARDS, COUPONS, or CUSTOM — organises vaults by campaign type
totalPoints	Total WP the vault can issue — fixed at creation
usedPoints	WP issued so far — incremented on each debit
conversionRateSnapshot	Rate locked at vault creation — all gift cards from this vault use this rate
sequenceOrder	When multiple vaults exist, lower number = higher priority for auto-selection
autoCreateOnThreshold	When usedPoints approaches totalPoints, automatically create the next vault
status	ACTIVE, EXHAUSTED (no points left), or DEACTIVATED

15.2 Gift Cards

Gift cards are single-use (or multi-use) redemption codes generated from a vault. When a customer redeems a gift card, the specified WP are credited to their wallet and debited from the source vault.

Gift Card Feature	Detail
code	24-character unique redemption code
wpValuePerUse	WP credited per redemption
maxUsages	How many times the code can be used (default: 1)
qualificationCriteria	JSON rules for who can redeem (e.g. employeeOnly, companyId)
isPublic	Whether non-employees can use a company-issued card
expiresAt	Optional expiry date
status	ACTIVE, EXHAUSTED, REVOKED, EXPIRED

15.3 Company Gift Cards

Companies can issue gift cards from their Company Wallet. This enables corporate benefit programs: a company tops up its corporate wallet and distributes gift card codes to employees, who redeem them for laundry orders.

i TECHNICAL NOTE: Gift card codes are pre-debited from their source (vault or company wallet) at creation time, not at redemption time. This prevents the source from being over-committed if multiple redemptions happen simultaneously.

16. Audit Trail & Ledger Immutability

16.1 The Three Ledgers

Ledger	Who It Belongs To	Purpose
LedgerEntry	Customer wallets	All WP credits and debits for customers (top-ups, order payments, refunds, gift cards)
VendorLedgerEntry	Vendor earnings wallets	All WP credits (earnings) and debits (payout requests) for vendors
RepPseudoLedgerEntry	Rep pseudo-wallets	All WP credits (delivery earnings) for reps — cycle resets included

16.2 Immutability Guarantee

All three ledgers share the same architectural pattern:

- No UPDATE or DELETE operations exist in the service layer for ledger records
- Every state change appends a NEW record with balanceBefore and balanceAfter
- The running balance on the wallet entity is a derived value — it should always equal the sum of all ledger entries (this is the reconciliation check)
- All ledger entities have a CreateDateColumn but NO UpdateDateColumn — there is no update path

16.3 Reconciliation

To verify wallet integrity:

```
SELECT SUM(CASE WHEN type = 'credit' THEN amount ELSE -amount END)
FROM    ledger_entries
WHERE   wallet_id = :walletId

-- Result must equal wallet.balance
-- Any discrepancy indicates data corruption or direct DB manipulation
```

17. Profile Completion Gate

Before a customer can place an order, the system checks that their profile is sufficiently complete. An incomplete profile is a common source of order failures — if we allow orders without a phone number, delivery cannot be coordinated; without an address, the Rep has nowhere to collect from.

17.1 Required Fields

Requirement	Check	Why It Matters
Phone number	user.phone is not null/empty	Rep calls customer to coordinate pickup; SMS notifications require a phone
Saved address	At least 1 Address record exists	Rep needs a physical location to collect from; required for area routing

17.2 What Happens on Failure

If either check fails, the system returns a structured 400 Bad Request:

```
{
  "message": "Profile incomplete — cannot place order",
  "missingFields": ["phone", "address"],
  "hint": "Add a phone number and at least one delivery address to your profile."
}
```

17.3 Profile Completion Endpoint

GET /users/me/profile-completion returns a checklist:

```
{
  "isComplete": false,
  "checks": { "phone": false, "address": true },
  "missingFields": ["phone"],
  "message": "Complete your profile to place orders. Missing: phone"
}
```

The app should call this endpoint on profile screens and display nudges to guide users to completion.

18. API Endpoint Reference

18.1 Pricing Endpoints

Method	Path	Auth	Description
POST	/pricing/calculate	Any	Authoritative price calculation for a specific order config
GET	/pricing/model/:areald	None	Full all-in pricing model for an area; cache ~5 min
GET	/pricing/config/:areald	None	Legacy alias for /pricing/model/:areald
GET	/pricing/packages	JWT required	List pricing packages visible to the authenticated user
GET	/pricing/intelligence	Admin/Finance	Full vendor price intelligence report
POST	/pricing/intelligence/apply	Admin only	Apply intelligence suggestions to platform price list
POST	/pricing/packages/admin	Admin only	Create a new pricing package
GET	/pricing/packages/admin	Admin/Finance	List all packages (including inactive)
GET	/pricing/packages/admin/:id	Admin/Finance	Get a single package
PATCH	/pricing/packages/admin/:id	Admin only	Update a package
DELETE	/pricing/packages/admin/:id	Admin only	Permanently delete a package

18.2 Platform Config Endpoints

Method	Path	Auth	Description
GET	/platform-config	Admin/Finance	Get current platform configuration
PATCH	/platform-config	Admin only	Update configuration fields (partial update)
POST	/platform-config/prices	Admin only	Add a new price list entry (bag, special_item, ironing)
GET	/platform-config/prices	Admin/Finance	List all price entries (newest first)
GET	/platform-config/bonus-tiers	Admin/Finance	List active Rep bonus tiers
POST	/platform-config/bonus-tiers	Admin only	Create or update a bonus tier
DELETE	/platform-config/bonus-tiers/:id	Admin only	Deactivate a bonus tier

18.3 User / Profile Endpoints (Financial Relevance)

Method	Path	Auth	Description
GET	/users/me/profile-completion	JWT required	Check if profile is complete for order placement
PATCH	/users/me	JWT required	Update phone number and other profile fields
POST	/users/me/addresses	JWT required	Add a delivery address (required before first order)

18.4 Order Endpoints (Financial)

Method	Path	Auth	Description
POST	/orders	JWT required	Place an order (profile gate + balance check + pricing + escrow creation)
PATCH	/orders/:id/confirm	JWT required	Customer confirms order complete → releases escrow

19. QA Test Scenarios

The following scenarios should be verified for every release. Each scenario includes the expected outcome and the database state to verify.

19.1 Pricing Engine Tests

Scenario	Input	Expected Result
Basic bag order	Medium Wash & Fold, no special items, 5% SC, 0% VAT, ₦150 transport	subtotalWP = bag price; serviceChargeWP = 5%; vatWP = 0; totalWP = subtotal+SC+transport
VAT enabled	Same as above but vatPercent = 7.5	vatWP = (subtotal + SC) × 7.5%; totalWP includes VAT
Special items	Small bag + 2 suits + 1 agbada	Line items for bag + each item; subtotal = sum of all; fees applied on subtotal
Wash & Iron	Large bag, wash_iron, ironingCount = 5	Ironing line item present at unitPriceWP × 5; subtotal includes ironing
Zero ironing on wash_fold	Medium bag, wash_fold, ironingCount = 10	Ironing line item NOT present (ironingCount ignored for wash_fold)
Transport area = 0	Area with transportFeeWP = 0	No transport line item; totalWP = subtotal + SC + VAT
Unknown special item	Item type not in price catalogue	Item skipped (unitPrice = 0); no line item added

19.2 Order Placement Tests

Scenario	Setup	Expected Outcome
Insufficient balance	User wallet = 500 WP; order total = 1,000 WP	400 Bad Request: "Insufficient WashPoints"
Missing phone (profile gate)	User has no phone, has address	400 Bad Request: missingFields = ["phone"]
Missing address (profile gate)	User has phone, no address	400 Bad Request: missingFields = ["address"]
Both missing	User has no phone and no address	400 Bad Request: missingFields = ["phone", "address"]
Successful order	Profile complete, sufficient balance	Order created; wallet debited; escrow created; ledger entry written; fiatBalanceKobo reduced proportionally
Correct fiatBalanceKobo deduction	Wallet: 10,000 WP, fiatBalance = 1,000,000 kobo; order = 2,000 WP	fiatBalance reduced by 20% → 800,000 kobo

19.3 Intelligence Engine Tests

Scenario	Setup	Expected Result
P70 calculation	10 vendors pricing suit: ₦600, ₦700, ₦750, ₦800, ₦850, ₦900, ₦950, ₦1000, ₦1100, ₦1200	P70 = ₦958.50 (linear interpolation on sorted array)
Only active pricing used	Vendor A has 2 pricing rows: old (expired) and new (active)	Only new row's items are included in analysis
Conversion to WP	suggestedNaira = ₦900, pointsPerUnit = 10	suggestedWP = 9,000
Apply with tolerance	Current platform price = 9,000 WP, suggested = 9,300 WP, toleranceWP = 500	Skipped (diff = 300 ≤ tolerance 500)
Apply creates new entry	Current = 7,500 WP, suggested = 9,000 WP, toleranceWP = 0	New platform_price_list entry created; old entry NOT deleted

19.4 Pricing Package Audience Tests

Scenario	Package Audience	User State	Visible?
allUsers package	{ allUsers: true }	Any user	Yes
minOrderCount = 5	{ minOrderCount: 5 }	User has 4 completed orders	No
minOrderCount = 5	{ minOrderCount: 5 }	User has 5 completed orders	Yes
Role check: corporate	{ roles: ["corporate"] }	User role = ["user"]	No
activeWithinDays = 30	{ activeWithinDays: 30 }	Last order was 45 days ago	No
requirePhone	{ requirePhone: true }	User has no phone	No
arealds match	{ arealds: ["uuid-lekki"] }	User's orders are in Lekki area	Yes
Package expired	validUntil = yesterday	Any user	No
Package not yet live	validFrom = tomorrow	Any user	No
Package inactive	isActive = false	Any user	No

19.5 Wallet & Ledger Integrity Tests

Test	Verification Method
Wallet balance = sum of ledger entries	SELECT SUM(CASE WHEN type='credit' THEN amount ELSE -amount END) FROM ledger_entries WHERE wallet_id = X
No ledger entry is ever updated	Check that no UpdateDateColumn exists on ledger entities; verify all DB rows have updated_at = created_at
fiatBalanceKobo never goes negative	After every order, assert wallet.fiatBalanceKobo >= 0
Payout rate snapshot frozen	Change payoutRateNairaPerWP after creating a payout request; verify request.payoutRateSnapshot unchanged

Conversion rate snapshot frozen on transaction	Change pointsPerUnit after initiating a Paystack transaction; verify transaction.conversionRateSnapshot unchanged
--	---

20. Glossary

WashPoints (WP): The internal currency of the Washermann platform. All prices, earnings, and transactions are denominated in WP.

Conversion Rate: The official exchange rate between Naira and WashPoints. Set by admin; multiple rates can coexist with different effective dates.

pointsPerUnit: WP per ₦1 — how many WashPoints a customer receives for each Naira spent.

Buy Rate: The customer-facing conversion rate (how much WP per Naira paid).

Payout Rate: The vendor/rep-facing rate (how much Naira per WP cashed out). Separate from buy rate.

Rate Spread: The difference between buy rate and payout rate — one component of platform margin.

PricingEngine: Stateless calculation class that converts inputs + config into a full PricingResult. No database access.

PricingConfig: The set of prices and percentages the engine needs — loaded from DB before each calculation.

PricingResult: The output of a PricingEngine.calculate() call — full itemised breakdown + totals.

lineItems: Individual charge lines in a PricingResult (bag, special item, ironing, service charge, VAT, transport).

Subtotal: Sum of item prices (bag + special items + ironing) before service charge, VAT, and transport.

Service Charge: A percentage fee applied on the order subtotal. Default 5%.

VAT: Value Added Tax — configurable percentage, default 0 (disabled). Applied on subtotal + service charge.

Transport Fee: Flat per-order fee set per Area. Covers the cost of Rep logistics.

Rep Share: Percentage of the order total allocated to the delivery Rep earnings pool. Default 15%.

Escrow: A financial hold on the customer's WP from order placement until order completion. Protects both parties.

WACB: Weighted Average Cost Basis — accounting method used to track the Naira cost basis of WP balances.

fiatBalanceKobo: Field on Wallet tracking the real Naira value of the WP balance, updated proportionally on each debit.

LedgerEntry: An immutable, append-only record of a single wallet transaction.

LedgerSource: Controlled enum identifying why a ledger entry was created (e.g. TOP_UP, ORDER_DEBIT, REFUND).

VendorEarningsWallet: Dedicated wallet for a vendor's accumulated WP earnings, separate from customer wallets.

RepPseudoWallet: Internal ops tool tracking Rep WP earnings. Not visible to the Rep — used to compute Naira salary.

Bonus Tier: Admin-configured rating band that determines a Rep's bonus percentage for the cycle.

Bonus Cycle: The period over which Rep earnings accumulate before salary calculation. Default: monthly.

Payout Request: Vendor's request to convert WP earnings to Naira bank transfer.

Vault: Admin-created pool of WashPoints used to fund gift cards and promotions.

Gift Card: Redemption code backed by a Vault or Company Wallet; credits WP to the redeemer's wallet.

Pricing Package: Promotional or niche bundle with a fixed WP price, structured criteria, and audience targeting rules.

Audience: JSON targeting rules on a PricingPackage defining who can see and use it.

Intelligence Engine: Service that aggregates vendor prices per garment type and computes statistical suggestions for platform pricing.

P70 / Percentile: Statistical measure. P70 means 70% of vendors price below this value. Used as the default platform price suggestion.

Platform Price List: Append-only catalogue of WashPoint prices for all item types. Never modified — only new rows added.

Profile Completion Gate: Mandatory check before order placement — requires phone number and at least one saved address.

Append-Only: Architectural pattern where records are never updated or deleted. State changes create new records.

This document is confidential and intended for authorised recipients only.

Washermann © 2026 — All Rights Reserved